Reviewer Pack — Zaslavsky Chamber Count Lower-Bounds AC⁰ PARITY Size

Entry 60678ef5bd8d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-16 23:34:23 UTC

Zaslavsky Chamber Count Lower-Bounds AC⁰ PARITY Size

Entry ID: `60678ef5bd8d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-16 23:34:23 UTC

1. Conjecture

Field A (mathematical branch): Hyperplane arrangement combinatorics (Zaslavsky chamber counting and Orlik-Solomon characteristic polynomial on the Boolean cube) **Field B** (complexity object): AC⁰ circuit size and depth for PARITY

Statement:

> For an AC⁰ circuit C with gate set G on n inputs, define the gate-arrangement A(C) by associating to each gate g (fanin set S_g, type t_g ∈ {AND,OR}) the affine hyperplane H_g: Σ_{i ∈ S_g} ε_{g,i} x_i = θ_g, where θ_g = |S_g|−1/2 for AND and 1/2 for OR and ε_{g,i} encodes literal polarity. Define the discrete chamber count ch(C) := |{ (1[g(x)=1])_{g ∈ G} : x ∈ {0,1}^n }|, i.e. the number of distinct gate-output covectors realized on the Boolean cube (equal up to a factor 2 to the Zaslavsky restricted chamber count of A(C) on {0,1}^n). Then for every depth-d ≥ 2 AC⁰ circuit C computing PARITY_n, $\log_2$ ch(C) ≥ (1/4) · n^{1/(d−1)}, and a single PARITY-computing circuit violating this bound at any (n,d) falsifies the conjecture.

Rationale (proposer's reasoning):

> Nisan–Wigderson designs control the pairwise intersection structure of gate input sets to force PRG fooling; chamber count of the gate-arrangement is precisely the structural quantity counting how many such intersection patterns must be jointly realized inside any circuit. Because chamber count is a relational, gate-structural invariant — not a poly-time predicate on truth tables — it sidesteps Razborov–Rudich, while still inheriting Hastad-style sensitivity through the Boolean-cube restriction. Unlike single-number truth-table invariants (Mahler, Plünnecke, magnitude, etc.) which have all hit naturalness, ch(C) tracks the categorical incidence between gates and inputs, mirroring the (S_i ∩ S_j) condition central to NW designs.

Taxonomy category: `NISAN_WIGDERSON_DESIGNS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ce67effdd780cd81`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across n∈{6,8,10,12,14,16} and 30 seeds, every PARITY-computing circuit in families (a) depth-2 DNF and (b) Hastad depth-3 must satisfy log2 ch(C) ≥ 0.25·n^{1/(d−1)}; support requires ≥28/30 seeds per (n,d) compliant with zero falsifiers, and mean negative-control ratio strictly below mean PARITY ratio.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `hyperplane arrangement AC0 circuit lower bound PARITY` - `Zaslavsky chamber count Boolean cube circuit complexity` - `covector count threshold circuits parity depth lower bound`

Top relevant hits considered: - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/1705.10753v2] Tutte Polynomials of Symmetric Hyperplane Arrangements - [http://arxiv.org/abs/math/0011101v3] Morse theory, Milnor fibers and minimality of hyperplane arrangements - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [http://arxiv.org/abs/1306.6132v3] Lattice Points in Orthotopes and a Huge Polynomial Tutte Invariant of Weighted Gain Graphs - [http://arxiv.org/abs/2311.04204v3] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import sys
import random
import math
import json
from collections import defaultdict

def generate_dnf_parity(n):
    circuit = []
    for i in range(2 ** (n - 1)):
        gate = [0] * n
        for j in range(n):
            if (i >> (n - 1 - j)) & 1:
                gate[j] = 1
        circuit.append(gate)
```

```
    return circuit

def generate_hastad_parity(n):
    if n == 1:
        return [[1]]
    m = int(math.sqrt(n))
    circuit = []
    for i in range(m):
        sub_circuit = generate_hastad_parity(m)
        for gate in sub_circuit:
            new_gate = [0] * n
            for j in range(len(gate)):
                if gate[j]:
                    new_gate[i * m + j] = 1
            circuit.append(new_gate)
    return circuit

def generate_random_dnf(n, size):
    circuit = []
    for _ in range(size):
        gate = [random.randint(0, 1) for _ in range(n)]
        circuit.append(gate)
    return circuit

def compute_ch(circuit, n):
    chamber_count = defaultdict(int)
    for x in range(2 ** n):
        output_vector = []
        for gate in circuit:
            if all(gate[i] == ((x >> (n - 1 - i)) & 1) for i in range(n)):
                output_vector.append(1)
            else:
                output_vector.append(0)
        chamber_count[tuple(output_vector)] += 1
    return len(chamber_count)

def run_trial(seed):
    random.seed(seed)
    n_values = [6, 8, 10, 12, 14, 16]
    results = []
    for n in n_values:
        # Generate PARITY circuits
        dnf_circuit = generate_dnf_parity(n)
        hastad_circuit = generate_hastad_parity(n)

        # Generate negative control
        size = len(dnf_circuit)
        random_circuit = generate_random_dnf(n, size)

        # Compute chamber counts
        dnf_ch = compute_ch(dnf_circuit, n)
        hastad_ch = compute_ch(hastad_circuit, n)
        random_ch = compute_ch(random_circuit, n)

        # Compute metric values
        dnf_metric = math.log2(dnf_ch) / (n ** (1 / 1))  # d=2
```

```
        hastad_metric = math.log2(hastad_ch) / (n ** (1 / 2))  # d=3
        random_metric = math.log2(random_ch) / (n ** (1 / 1))  # d=2

        # Check conjecture
        dnf_holds = dnf_metric >= 0.25
        hastad_holds = hastad_metric >= 0.25
        random_holds = random_metric < dnf_metric

        # Prepare results
        results.append({
            "n": n,
            "dnf_metric": dnf_metric,
            "hastad_metric": hastad_metric,
            "random_metric": random_metric,
            "dnf_holds": dnf_holds,
            "hastad_holds": hastad_holds,
            "random_holds": random_holds,
            "counterexample": "" if dnf_holds and hastad_holds else f"n={n}, d=2, size={len(dnf_ci
        })

    # Aggregate results
    dnf_metrics = [r["dnf_metric"] for r in results]
    hastad_metrics = [r["hastad_metric"] for r in results]
    random_metrics = [r["random_metric"] for r in results]

    dnf_holds = all(r["dnf_holds"] for r in results)
    hastad_holds = all(r["hastad_holds"] for r in results)
    random_holds = all(r["random_holds"] for r in results)

    counterexamples = [r["counterexample"] for r in results if r["counterexample"]]

    return {
        "metric_name": "log2(ch(C)) / n^{1/(d-1)}",
        "metric_value": sum(dnf_metrics + hastad_metrics) / (2 * len(n_values)),
        "instances_tested": len(n_values),
        "conjecture_holds": dnf_holds and hastad_holds and random_holds,
        "counterexample": counterexamples[0] if counterexamples else ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    trials = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps({'seed': seed, **result})}")
        trials.append(result)

    # Compute statistics
    metric_values = [t["metric_value"] for t in trials]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)
    support_fraction = sum(1 for t in trials if t["conjecture_holds"]) / len(trials)

    # Determine result
    if all(t["conjecture_holds"] for t in trials):
        print(f"RESULT: SUPPORTED mean={mean_metric:.4f} std={std_metric:.4f} support_fraction={su
```

```
    elif any(t["counterexample"] for t in trials):
        first_failing_seed = next(t["seed"] for t in trials if t["counterexample"])
        print(f"RESULT: FALSIFIED counterexample=\"{trials[0]['counterexample']}\" first_failing_s
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_support")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out after 240s (returncode=124) without producing any RESULT line or seed-level data, so the pre-registered support condition cannot be evaluated and the critic's CHALLENGE stands. | next: Reduce n range (e.g., n∈{6,8,10}) and/or seed count, and optimize chamber enumeration (cache covectors via bitsets) to bring runtime under the 240s budget before retrying.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	272765
2	preregistration	claude_max	opus	0	0	7228
3	novelty	claude_max	opus	0	0	3716
4	novelty	claude_max	opus	0	0	9851
5	test_gen	mistral	codestral-latest	0	0	311169
6	test_gen	mistral	codestral-latest	0	0	312158
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	278818
8	test_gen	mistral	codestral-latest	0	0	310489
9	judge	claude_max	opus	0	0	4352

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 1510548 ms total latency. Provider mix: {'claude_max': 5, 'mistral': 3, 'ollama_remote': 1}

(full prompt+response transcripts available in `research/audit/60678ef5bd8d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/60678ef5bd8d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/60678ef5bd8d.tar.gz` (if generated)